Rapport de TP : premiers déploiements Kubernetes avec Minikube et Docker

Binôme : Adam Beloucif et Emilien Morice

Formation : EFREI, M2 Data Engineering & IA, module DevOps

Date d'exécution : 19 octobre 2026

Dépôt : https://github.com/Adam-Blf/tp-kubernetes-minikube

Introduction

Ce TP fait passer une application de l'état de conteneur isolé à celui de charge gérée par Kubernetes. On démarre un cluster local avec Minikube, puis on compare un Pod autonome à un Pod géré par un Deployment. On observe ensuite l'auto-réparation et la mise à l'échelle, on expose l'application par un Service, et on finit par déployer une image construite soi-même.

Toutes les captures de ce rapport proviennent d'une même exécution complète du TP, rejouée le 19/10/2026 par scripts/rejouer_tp.sh. Chaque capture est le rendu fidèle de la sortie brute conservée à côté d'elle dans preuves/*.txt.

Environnement

Outil	Version
Docker (client et serveur)	29.8.2
kubectl (client)	v1.36.1
Minikube	v1.39.0
Kubernetes (nœud)	v1.37.0, runtime containerd 2.3.4
Système	Windows 11 Pro, Docker Desktop (WSL2), driver docker

Démarrer et vérifier le cluster (§3)

Commandes : docker version, kubectl version --client, minikube version, minikube start --driver=docker, minikube status, docker ps, kubectl cluster-info, kubectl get nodes -o wide, kubectl get pods -A.

terminal - minikube (namespace demo)

Figure 1 - Démarrage et inspection du cluster Minikube (versions, statut, nœud, pods système)

```
$ docker version
Client:
 Version:           29.8.2
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        7fc2dff
 Built:             Wed Sep 30 19:35:42 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.94.0 (241994)
 Engine:
  Version:          29.8.2
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.8
  Git commit:       8af9fe3
  Built:            Wed Sep 30 19:32:25 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.3.6
  GitCommit:        ee2735368117d2eb259779949d5e75cdafec9761
 runc:
  Version:          1.5.1
  GitCommit:        v1.5.1-0-g8f2685a4
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0

$ kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1

$ minikube version
minikube version: v1.39.0
commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10

$ minikube start --driver=docker --memory=3072 --cpus=2
* minikube v1.39.0 on Microsoft Windows 11 Pro 25H2
* Using the docker driver based on existing profile
! You cannot change the memory size for an existing minikube cluster. Please first delete the cluster.
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured


$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES                   IMAGE                                       STATUS
sales-sonarqube         sonarqube:community                         Up 2 minutes
sales-sonarqube-db      postgres:16-alpine                          Up 2 minutes
sales-spark-streaming   real-time-sales-devops-tp-spark-streaming   Up 4 minutes
sales-kafka             confluentinc/cp-kafka:7.6.14                Up 5 minutes (healthy)
sales-api               real-time-sales-devops-tp-sales-api         Up 15 minutes
sales-spark-worker      apache/spark:3.5.7                          Up 15 minutes
sales-postgres          postgres:16-alpine                          Up 15 minutes (healthy)
sales-spark-master      apache/spark:3.5.7                          Up 15 minutes
sales-zookeeper         confluentinc/cp-zookeeper:7.6.14            Up 15 minutes
minikube                gcr.io/k8s-minikube/kicbase:v0.0.51         Up 37 seconds

$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:65023
CoreDNS is running at https://127.0.0.1:65023/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

$ kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE    VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION
    CONTAINER-RUNTIME
minikube   Ready    control-plane   154m   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)
    6.6.87.2-microsoft-standard-WSL2 (amd64)   containerd://2.3.4

$ kubectl get pods -A
NAMESPACE     NAME                               READY   STATUS    RESTARTS      AGE
default       api-deployment-cbd88b68d-bwfpr     1/1     Running   2 (32s ago)   149m
default       api-deployment-cbd88b68d-mb5hz     1/1     Running   2 (32s ago)   149m
default       api-deployment-cbd88b68d-pwntj     1/1     Running   2 (32s ago)   149m
default       db-7499c9cfff-vg6k4                1/1     Running   2 (32s ago)   150m
kube-system   coredns-559f6c778d-wmzmz           0/1     Running   2 (32s ago)   154m
kube-system   etcd-minikube                      1/1     Running   2 (32s ago)   154m
kube-system   kindnet-ptktk                      1/1     Running   2 (32s ago)   154m
kube-system   kube-apiserver-minikube            1/1     Running   2 (32s ago)   154m
kube-system   kube-controller-manager-minikube   1/1     Running   2 (32s ago)   154m
kube-system   kube-proxy-7lglk                   1/1     Running   2 (32s ago)   154m
kube-system   kube-scheduler-minikube            1/1     Running   2 (32s ago)   154m
kube-system   storage-provisioner                1/1     Running   3 (32s ago)   154m
```

Nœud. Le cluster compte un seul nœud, minikube, de rôle control-plane, au statut Ready, en Kubernetes v1.37.0 avec le runtime containerd. Avec le driver docker, ce nœud est lui-même un conteneur Docker, visible dans docker ps. Il porte à la fois le plan de contrôle et les charges applicatives.

Trois composants système observés dans le namespace kube-system :

- kube-apiserver-minikube : point d'entrée de l'API, par lequel passent kubectl et tous les autres composants ;
- etcd-minikube : base clé-valeur qui stocke l'état du cluster ;
- coredns : DNS interne qui résout les noms des Services.

On voit aussi kube-scheduler, kube-controller-manager, kube-proxy, kindnet (réseau) et storage-provisioner.

Le profil Minikube existait déjà. minikube start le réutilise et signale qu'on ne peut pas changer sa taille mémoire sans le supprimer, ce qui n'empêche pas le démarrage.

Namespace et Pod autonome (§4)

On crée le namespace demo, on en fait le namespace courant, puis on applique manifests/pod.yaml : un Pod whoami-pod, image traefik/whoami:v1.10.3, port 80.

terminal - minikube (namespace demo)

Figure 6 - Pod autonome : inspection, hostname, suppression sans recréation

```
$ kubectl create namespace demo
namespace/demo created

$ kubectl config set-context --current --namespace=demo
Context "minikube" modified.

$ kubectl apply -f manifests/pod.yaml
pod/whoami-pod created

$ kubectl wait --for=condition=Ready pod/whoami-pod --timeout=180s
pod/whoami-pod condition met

$ kubectl get pods -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
whoami-pod   1/1     Running   0          3s    10.244.0.7   minikube   <none>           <none>

$ kubectl describe pod whoami-pod | head -n 30
Name:             whoami-pod
Namespace:        demo
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Mon, 19 Oct 2026 14:46:45 +0200
Labels:           app=whoami-pod
Annotations:      <none>
Status:           Running
IP:               10.244.0.7
IPs:
  IP:  10.244.0.7
Containers:
  whoami:
    Container ID:   containerd://27ded6ea93eaed042827f92ed056b42710fa29f19d2f5a1d06c610c8b577b9a5
    Image:          traefik/whoami:v1.10.3
    Image ID:       docker.io/traefik/whoami@sha256:43a68d10b9dfcfc3ffbfe4dd42100dc9aeaf29b3a5636c856337a5940f1b4f1c
    Port:           80/TCP (http)
    Host Port:      0/TCP (http)
    State:          Running
      Started:      Mon, 19 Oct 2026 14:46:47 +0200
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-bgqln (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True

$ kubectl logs whoami-pod
2026/10/19 12:46:47 Starting up on port 80

$ kubectl exec whoami-pod -- sh -c hostname
error: Internal error occurred: Internal error occurred: error executing command in container: failed to exec in container: failed
    to start exec "0d06544c85a3bfb12d9916cd55537f49ccd618adab9817a9aac630510d6ac291": OCI runtime exec failed: exec failed: unable
    to start container process: exec: "sh": executable file not found in $PATH

# traefik/whoami ne contient ni shell ni binaire hostname : on attache un conteneur de débogage busybox qui partage l'espace de
    noms du Pod.

$ kubectl debug whoami-pod -q --image=busybox:1.36 --target=whoami --attach=true -- hostname
warning: couldn't attach to pod/whoami-pod, falling back to streaming logs: Internal error occurred: Internal error occurred:
    error attaching to container: failed to load task: no running task found: task
    fd4dcfe7d9bdc3ba03845b33c4c49b617f904246bb1771f7039654df59031006 not found
whoami-pod

$ kubectl delete pod whoami-pod
pod "whoami-pod" deleted from demo namespace

$ kubectl get pods
No resources found in demo namespace.

# Le Pod autonome ne revient pas : aucun contrôleur ne le surveille.
```

get, describe et logs montrent un Pod Running qui écoute sur le port 80.

Le hostname dans le conteneur. kubectl exec -it whoami-pod -- sh échoue : l'image traefik/whoami est minimale et ne contient ni shell ni binaire hostname. On attache donc un conteneur de débogage éphémère busybox qui partage l'espace de noms du Pod (kubectl debug ... --target=whoami -- hostname), et il renvoie bien whoami-pod. L'avertissement affiché vient de ce que la commande se termine avant que kubectl ait pu s'y attacher ; il bascule alors sur les journaux, qui contiennent la réponse.

Suppression. Après kubectl delete pod whoami-pod, kubectl get pods ne renvoie plus rien : aucun contrôleur ne possède ce Pod, il ne revient pas.

Le Pod porte le label app: whoami-pod et non app: whoami : sinon le Service de la partie 6 le sélectionnerait aussi.

Deployment à trois réplicas (§5)

manifests/deployment.yaml décrit un Deployment apps/v1 nommé whoami, à 3 réplicas. Son spec.selector.matchLabels (app: whoami) correspond aux labels du modèle de Pod.

terminal - minikube (namespace demo)

Figure 2 - Deployment whoami : 3 réplicas, ReplicaSet, labels et describe

```
$ kubectl apply -f manifests/deployment.yaml
deployment.apps/whoami created

$ kubectl rollout status deployment/whoami --timeout=180s
Waiting for deployment "whoami" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "whoami" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "whoami" rollout to finish: 2 of 3 updated replicas are available...
deployment "whoami" successfully rolled out

$ kubectl get deployment,replicaset,pods
NAME                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/whoami   3/3     3            3           2s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/whoami-c6f799d48    3         3         3       2s

NAME                          READY   STATUS    RESTARTS   AGE
pod/whoami-c6f799d48-cdvzq    1/1     Running   0          2s
pod/whoami-c6f799d48-vz8df    1/1     Running   0          2s
pod/whoami-c6f799d48-w4hdh    1/1     Running   0          2s

$ kubectl get pods --show-labels
NAME                     READY   STATUS    RESTARTS   AGE   LABELS
whoami-c6f799d48-cdvzq   1/1     Running   0          3s    app=whoami,pod-template-hash=c6f799d48
whoami-c6f799d48-vz8df   1/1     Running   0          3s    app=whoami,pod-template-hash=c6f799d48
whoami-c6f799d48-w4hdh   1/1     Running   0          3s    app=whoami,pod-template-hash=c6f799d48

$ kubectl describe deployment whoami | head -n 30
Name:                   whoami
Namespace:              demo
CreationTimestamp:      Mon, 19 Oct 2026 14:46:58 +0200
Labels:                 app=whoami
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=whoami
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=whoami
  Containers:
   whoami:
    Image:         traefik/whoami:v1.10.3
    Port:          80/TCP (http)
    Host Port:     0/TCP (http)
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   whoami-c6f799d48 (3/3 replicas created)
Events:
```

kubectl get deployment,replicaset,pods montre la chaîne de possession : le Deployment whoami crée le ReplicaSet whoami-c6f799d48, qui crée trois Pods nommés d'après lui. --show-labels confirme que chaque Pod porte app=whoami et le pod-template-hash du ReplicaSet.

Auto-réparation (§5)

On supprime un Pod du Deployment par son nom réel pendant qu'un second terminal observe avec kubectl get pods -w.



Juste après la suppression, le Pod cdvzq est en Terminating à côté de son remplaçant dfhhb en ContainerCreating. Le flux d'événements donne l'ordre exact : le Pod supprimé passe en Terminating, le ReplicaSet crée aussitôt dfhhb (Pending, ContainerCreating, Running en 2 secondes), et l'ancien Pod est retiré. On revient à trois Pods Running.

Mise à l'échelle (§5)

terminal - minikube (namespace demo)

Figure 3b - Mise à l'échelle : 3 puis 5, 0 et retour à 3 Pods

```
$ kubectl scale deployment whoami --replicas=5
deployment.apps/whoami scaled

$ kubectl rollout status deployment/whoami --timeout=120s
Waiting for deployment "whoami" rollout to finish: 3 of 5 updated replicas are available...
Waiting for deployment "whoami" rollout to finish: 4 of 5 updated replicas are available...
deployment "whoami" successfully rolled out

$ kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
whoami-c6f799d48-48xfk    1/1     Running   0          2s
whoami-c6f799d48-cfzzv    1/1     Running   0          2s
whoami-c6f799d48-dfhhb    1/1     Running   0          16s
whoami-c6f799d48-vz8df    1/1     Running   0          24s
whoami-c6f799d48-w4hdh    1/1     Running   0          24s

$ kubectl scale deployment whoami --replicas=0
deployment.apps/whoami scaled

$ kubectl get pods
NAME                      READY   STATUS        RESTARTS   AGE
whoami-c6f799d48-48xfk    1/1     Terminating   0          3s
whoami-c6f799d48-cfzzv    1/1     Terminating   0          3s
whoami-c6f799d48-dfhhb    1/1     Terminating   0          17s
whoami-c6f799d48-vz8df    1/1     Terminating   0          25s
whoami-c6f799d48-w4hdh    1/1     Terminating   0          25s

$ kubectl get pods
No resources found in demo namespace.

$ kubectl scale deployment whoami --replicas=3
deployment.apps/whoami scaled

$ kubectl rollout status deployment/whoami --timeout=120s
Waiting for deployment "whoami" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "whoami" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "whoami" rollout to finish: 2 of 3 updated replicas are available...
deployment "whoami" successfully rolled out

$ kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
whoami-c6f799d48-29hc6    1/1     Running   0          2s
whoami-c6f799d48-kq588    1/1     Running   0          2s
whoami-c6f799d48-l7wqf    1/1     Running   0          2s
```

- **5 réplicas** : deux Pods sont ajoutés, les trois existants sont conservés.
- **0 réplica** : les cinq Pods passent en Terminating, puis plus aucun Pod n'existe ; le Deployment et son ReplicaSet restent.
- **3 réplicas** : trois nouveaux Pods sont créés et rollout status confirme la fin du déploiement.

Service (§6)

manifests/service.yaml décrit un Service whoami-service de type ClusterIP, qui sélectionne app: whoami et envoie le port 80 vers le port nommé http des conteneurs.

terminal - minikube (namespace demo)

Figure 4 - Service whoami-service, endpoints et réponse HTTP via port-forward 8080

```
$ kubectl apply -f manifests/service.yaml
service/whoami-service created

$ kubectl get services
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
whoami-service   ClusterIP   10.96.231.156   <none>        80/TCP    0s

$ kubectl get endpoints whoami-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME             ENDPOINTS                                      AGE
whoami-service   10.244.0.14:80,10.244.0.15:80,10.244.0.16:80   0s

$ kubectl get pods -o wide
NAME                      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
whoami-c6f799d48-29hc6    1/1     Running   0          3s    10.244.0.16   minikube   <none>           <none>
whoami-c6f799d48-kq588    1/1     Running   0          3s    10.244.0.14   minikube   <none>           <none>
whoami-c6f799d48-l7wqf    1/1     Running   0          3s    10.244.0.15   minikube   <none>           <none>

$ curl -s http://localhost:8080
Hostname: whoami-c6f799d48-29hc6
IP: 127.0.0.1
IP: ::1
IP: 10.244.0.16
IP: fe80::cce8:38ff:fe14:6542
RemoteAddr: 127.0.0.1:57720
GET / HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.18.0
Accept: */*
```

Les trois endpoints (10.244.0.14, .15, .16 sur le port 80) sont exactement les IP des trois Pods données par kubectl get pods -o wide : le Service cible les bons Pods. Via kubectl port-forward service/whoami-service 8080:80, curl reçoit la réponse de whoami, dont la ligne Hostname porte le nom d'un Pod du Deployment.

Image personnalisée (§7)

monapp/index.html contient un titre et un message en français, et monapp/Dockerfile part de nginx:1.27 et copie la page dans /usr/share/nginx/html/.

terminal - minikube (namespace demo)

Figure 5 - Image monapp:1.0 chargée dans Minikube, Deployment et page servie (8081)

```
$ docker build -t monapp:1.0 ./monapp 2>&1 | tail -n 12
#6 CACHED

#7 exporting to image
#7 exporting layers done
#7 exporting manifest sha256:962edb01a38286f51408ff8921d1326972cf2b03d3ff910f3dec15534e81559c done
#7 exporting config sha256:0bcd16355240f765cf2107f196fee6e609a5d92b8ba91778ed98d1a9a2c286c3 done
#7 exporting attestation manifest sha256:b9b1748871d183da3fc24ee0e268a61207b7374a314fef19f328de07bca0ff0a 0.0s done
#7 exporting manifest list sha256:9188c124b6dd0d4f85108d62f99c9d8ec488cb05eb06ccb23432a6737f37235d 0.0s done
#7 naming to docker.io/library/monapp:1.0
#7 naming to docker.io/library/monapp:1.0 done
#7 unpacking to docker.io/library/monapp:1.0 0.0s done
#7 DONE 0.2s

$ minikube image load monapp:1.0

$ minikube image ls | grep monapp
docker.io/library/monapp:1.0

$ kubectl apply -f monapp-deployment.yaml
deployment.apps/monapp created

$ kubectl rollout status deployment/monapp --timeout=120s
Waiting for deployment "monapp" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "monapp" rollout to finish: 1 of 2 updated replicas are available...
deployment "monapp" successfully rolled out

$ kubectl get pods -l app=monapp
NAME                      READY   STATUS    RESTARTS   AGE
monapp-577998cd66-62ktp   1/1     Running   0          9s
monapp-577998cd66-tx2fc   1/1     Running   0          9s

$ curl -s http://localhost:8081
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Mon application sur Kubernetes</title>
</head>
<body>
  <h1>Mon application sur Kubernetes</h1>
  <p>Bonjour ! Cette page est servie par un conteneur Nginx déployé avec Minikube.</p>
</body>
</html>
```

docker build produit monapp:1.0, minikube image load la copie dans le nœud (minikube image ls la liste), puis monapp-deployment.yaml la déploie sur 2 réplicas avec imagePullPolicy: Never. Les deux Pods sont Running et la page en français est servie sur localhost:8081.

Nettoyage (§10)

terminal - minikube (namespace demo)

Figure 7 - Nettoyage : ressources et namespace demo supprimés, minikube stop

```
$ kubectl delete -f manifests/service.yaml --ignore-not-found
service "whoami-service" deleted from demo namespace

$ kubectl delete -f manifests/deployment.yaml --ignore-not-found
deployment.apps "whoami" deleted from demo namespace

$ kubectl delete -f manifests/pod.yaml --ignore-not-found

$ kubectl delete -f monapp-deployment.yaml --ignore-not-found
deployment.apps "monapp" deleted from demo namespace

$ kubectl delete namespace demo
namespace "demo" deleted

$ kubectl config set-context --current --namespace=default
Context "minikube" modified.

$ minikube stop
* Stopping node "minikube"  ...
* Powering off "minikube" via SSH ...
* 1 node stopped.
```

Toutes les ressources sont supprimées, ainsi que le namespace demo. On revient au namespace default et minikube stop arrête le nœud. Le cluster est conservé et pourra redémarrer sans tout retélécharger.

Questions de synthèse (§8)

Pod autonome et Pod géré par un Deployment. Le Pod autonome n'a aucun propriétaire : supprimé ou planté, il est perdu (figure 6). Le Pod géré par un Deployment appartient à un ReplicaSet qui maintient le nombre de réplicas voulu (figure 3). Le Deployment apporte aussi la mise à l'échelle et les mises à jour progressives avec retour arrière.

Rôle du ReplicaSet dans l'auto-réparation. Il compare en continu le nombre de Pods qui correspondent à son sélecteur au nombre désiré. Dès qu'il voit un écart, il crée ou supprime des Pods pour revenir à l'état désiré : c'est la boucle de réconciliation. Le même mécanisme joue pour un Pod supprimé et pour un scale.

Rôle du Service et vérification. Les Pods sont éphémères et changent d'IP à chaque recréation. Le Service offre une IP et un nom DNS stables, et répartit le trafic entre les Pods dont les labels correspondent à son sélecteur. Pour vérifier qu'il cible les bons Pods, on compare ses endpoints aux IP de kubectl get pods -o wide, on contrôle les labels avec --show-labels, et on teste une requête. Si les endpoints sont vides, le sélecteur ne correspond à aucun Pod prêt.

Image chargée dans Minikube et imagePullPolicy: Never. Le nœud Minikube a son propre runtime et son propre stock d'images, séparé du Docker de l'hôte. monapp:1.0 n'existe qu'en local et n'est sur aucun registre : minikube image load la copie dans le nœud. imagePullPolicy: Never force kubelet à utiliser cette copie sans tenter de la télécharger. Sans cela, il la chercherait sur Docker Hub et le Pod finirait en ErrImagePull ou en ImagePullBackOff.

Difficultés rencontrées

- **Pas de shell dans traefik/whoami** : résolu par un conteneur de débogage éphémère.
- **Minikube sur un profil existant** : redémarrage long la première fois, et taille mémoire non modifiable sans minikube delete.
- **Machine partagée** : une stack Kafka, Spark et Postgres d'un autre TP tournait en parallèle.
- **port-forward** : il s'attache à un seul Pod, donc la répartition du Service ne s'observe pas par curl à travers lui.
- **Endpoints dépréciés** : kubectl get endpoints avertit qu'EndpointSlice le remplace depuis Kubernetes 1.33.

Répartition dans le binôme

Emilien Morice a préparé la partie Pod autonome : la démarche, la conclusion et une première version de pod.yaml, reprises dans la mini doc docs/pod-autonome.md. Adam Beloucif a réalisé le cluster, le Deployment, le Service, l'image personnalisée, les preuves, l'intégration continue et ce rapport.

Conclusion

Le TP montre ce que Kubernetes ajoute à Docker : un état désiré déclaré dans des manifestes, et des contrôleurs qui le maintiennent. Un Pod seul ne vaut guère mieux qu'un docker run ; avec un Deployment, la perte d'un Pod se répare en quelques secondes et la capacité change d'une commande. Le Service découple les clients de la vie des Pods. Enfin, le cas de l'image locale rappelle que le nœud est une machine distincte, avec son propre stock d'images.

Annexe : manifestes

manifests/pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: whoami-pod
  namespace: demo
  labels:
    app: whoami-pod
spec:
  containers:
    - name: whoami
      image: traefik/whoami:v1.10.3
      ports:
        - containerPort: 80
          name: http
```

manifests/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: whoami
```

```
  namespace: demo
  labels:
    app: whoami
spec:
  replicas: 3
  selector:
    matchLabels:
      app: whoami
  template:
    metadata:
      labels:
        app: whoami
    spec:
      containers:
        - name: whoami
          image: traefik/whoami:v1.10.3
          ports:
            - containerPort: 80
              name: http
```

manifests/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: whoami-service
  namespace: demo
spec:
  type: ClusterIP
  selector:
    app: whoami
  ports:
    - name: http
      port: 80
      targetPort: http
```

monapp-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: monapp
  namespace: demo
  labels:
    app: monapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: monapp
  template:
    metadata:
      labels:
        app: monapp
    spec:
      containers:
        - name: monapp
          image: monapp:1.0
          imagePullPolicy: Never
          ports:
```

```
            - containerPort: 80
              name: http
```

monapp/Dockerfile

```
FROM nginx:1.27
COPY index.html /usr/share/nginx/html/index.html
```